

Assignment 3

“You must also address any comments you received for Assignment #2. You should update your model accordingly if the comment was about the architecture. Any comments that have not been addressed will result in the exact deduction you received for Assignment #2.”

Assignment 2 comments: - NONE. Got no comments on Assignment 2.

Framework Chosen: - FastAPI + HTMX + Jinja(server-rendered UI), SQLite, diskcache (LRU + TTL), llama.cpp (GGUF, CPU-only)

Rationale:

Our App fixes CPU-only on a small VM, quantized 1-8B models, REST + SSE, strict local caching, RBAC/admin/API keys, TLS, and easy deployment. Hence,

FastAPI/Starlette gives typed REST and native SSE with low overhead.

Llama.cpp meets CPU-only and memory limits.

Diskcache delivers L:RU + TTL and hit/miss metrics for the admin console.

SQLite keeps sessions, users, and keys portable.

HTMX + Jinja provides an accessible UI without SPA weight.

This tech stack directly satisfies my Assignment 2 functional and non-functional items.

The architecture in my Assignment 2 remains unchanged in scope and constraints, but in this assignment, I refine the same system into the FastAPI + HTMX + Jinja framework by making each abstract element concrete. The model now defines routers, services, adapters, DTOs and templates explicitly, making each interaction implementable within the framework's routing, dependency-injection, and lifespan model.

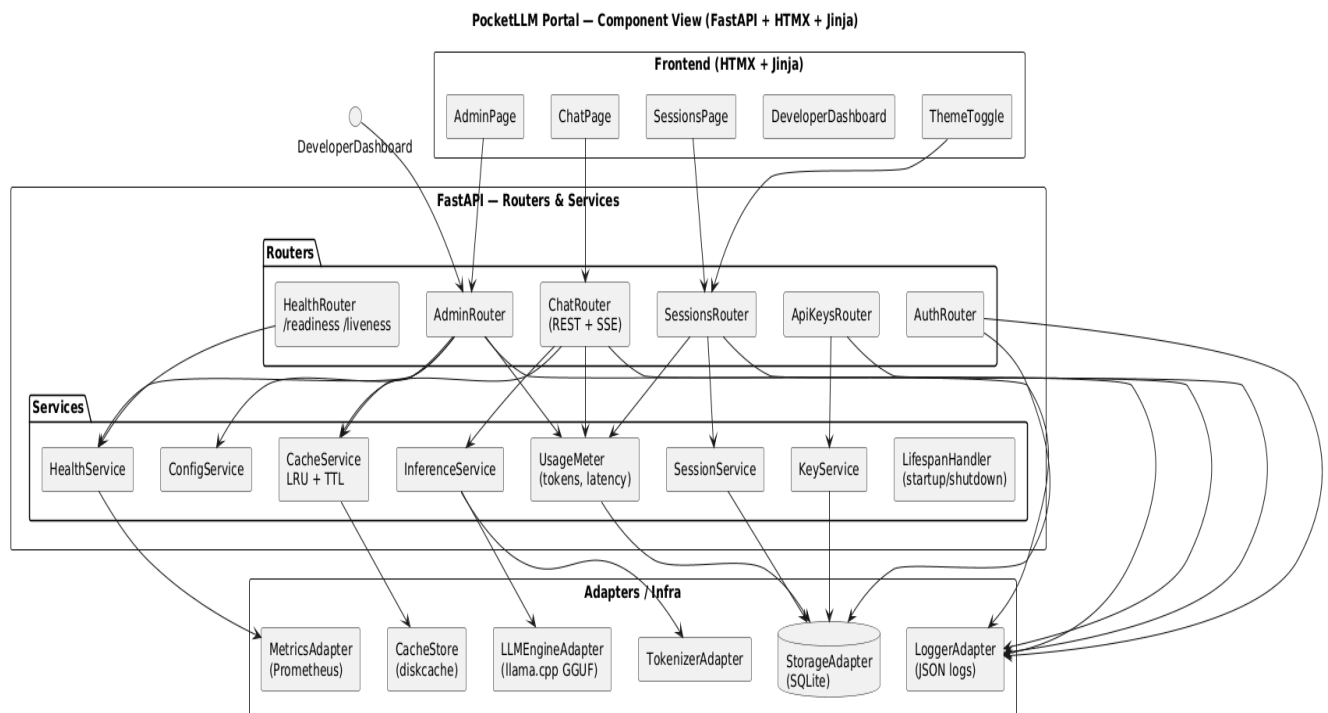
Changes/Departures:

1. **Routers instead of API Endpoints:** Explicit FastAPI routers replace the generic API Endpoints in Assignment 2. FastAPI organizes endpoints through routers. It removes ambiguity about where each REST and SSE route lives.
2. **Typed DTOs:** Framework expects explicit request/response models. Adding PromptRequest and PromptResponse avoids parameter usage guessing. This clarifies parameter control and usage reporting.
3. **Auth dependency guard:** Adding dependency guard with roles using idiomatic FastAPI DI guards via dependency injection will provide concrete protection for Admin/Keys routes.
4. **Disk-backed cache (LRU + TTL):** Split into CacheService + disk store. On disk store handles persistence. Service adds keys, TTL and stats. This enables admin cache stats and <1s hit path.
5. **Lifespan handler:** Use startup/shutdown events to drain in-flight requests, persist cache index and unload GGUF. This satisfies the graceful shutdown requirement from the previous assignment.
6. **Health endpoints:** Add HealthRouter to satisfy System Monitoring and health checks part from previous assignment. This removes guessing work about where the health endpoint lives and what it returns.
7. **Transport/TLS placement:** Annotate that HTTPS terminates at a reverse proxy(443) and forwards to the FastAPI app(8000). That shows explicit compliance with Transport Security part in our previous assignment.

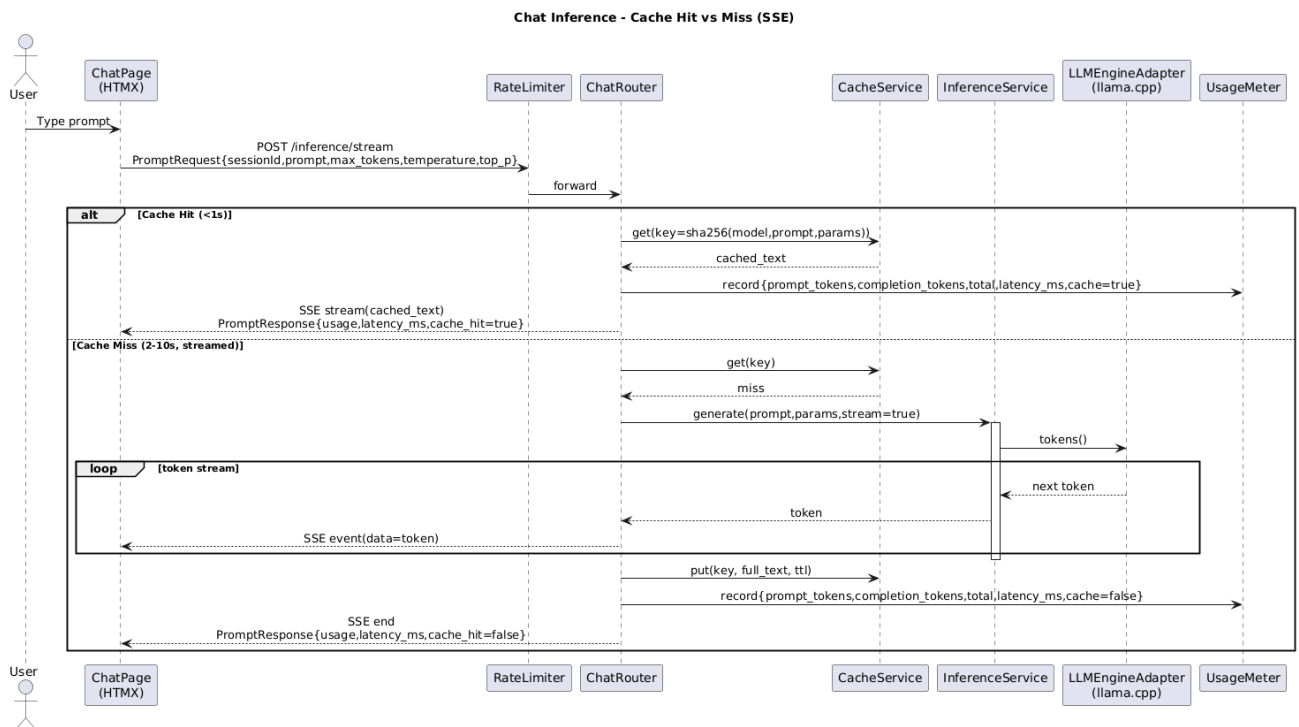
UML Diagrams:

A. Updated from Assignment 2:

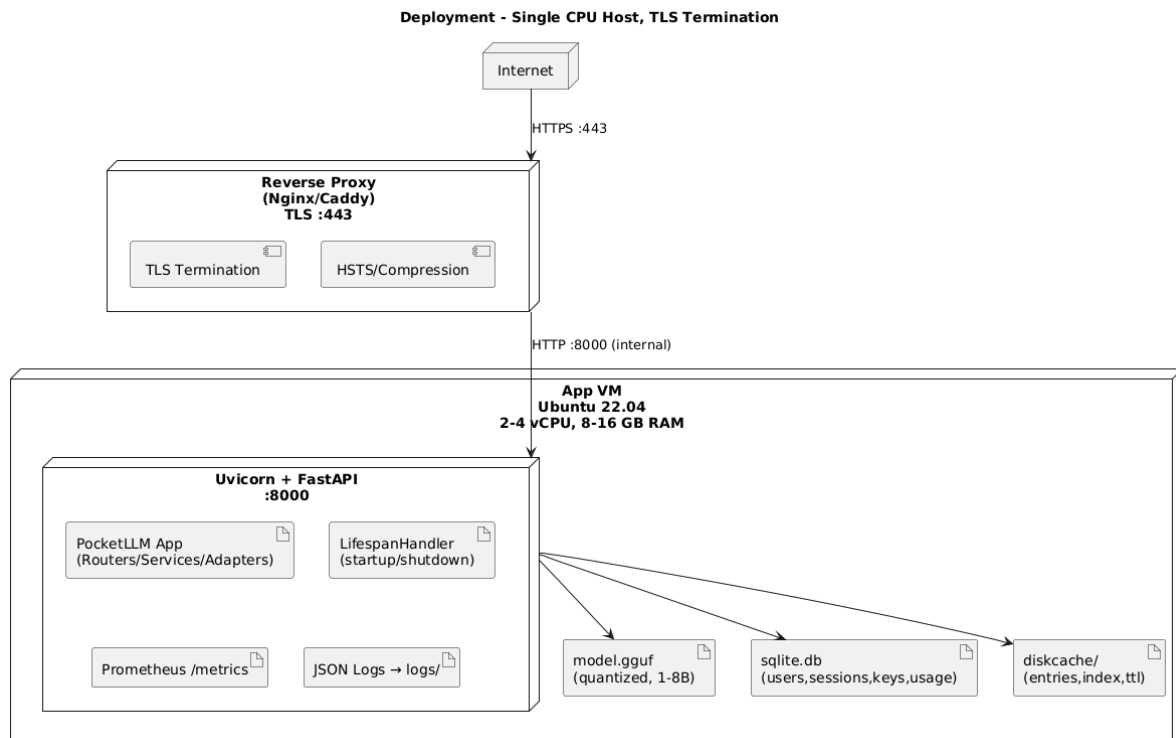
1. Component Diagram – Updated to be Framework Aware



2. Sequence Diagram – Updated to have Chat (cache hit vs miss, SSE)

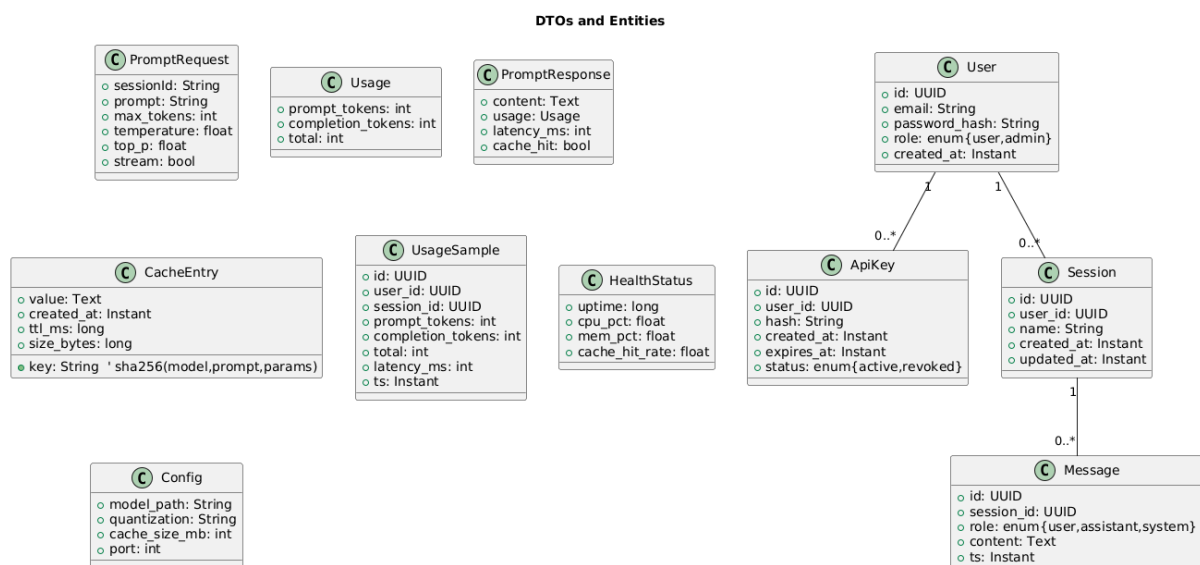


3. Deployment Diagram – Updated for CPU-only, TLS 443 → 8000

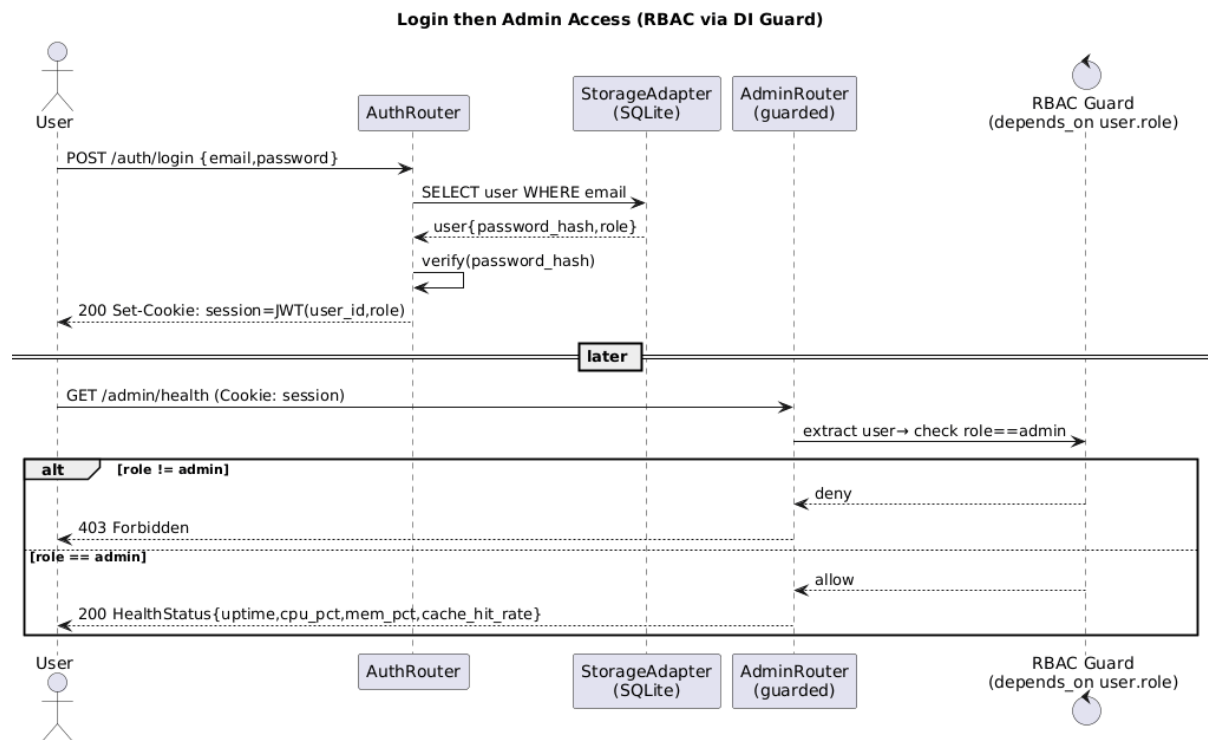


B. New Diagrams for completeness:

1. Class Diagram — DTOs and Entities



2. Sequence Diagram — Login → RBAC guard (Admin)



3. Sequence Diagram — Graceful Shutdown (lifespan)

